

CogniFit Escolar — Diseño del Backend (Entregables 1, 2 y 4)

Rodrigo Emilio Campuzano Culebro — Backend SOA / Base de Datos

Universidad Politécnica de Chiapas — Ingeniería en Software

- [Diseño del Backend — Implementación ↔ Patrón ↔ Requerimiento \(Entregable #1\)](#)
 - [1. Arquitectura general](#)
 - [2. Mapa Implementación ↔ Patrón ↔ ADR/HU](#)
 - [3. Modelo de datos \(BD\) y HU cubiertas](#)
 - [4. Seguridad \(OWASP\)](#)
 - [5. Integración con microservicios ML \(Carlos\)](#)
 - [6. Estado de avance](#)
- [Diseño Visual de la API — CogniFit Backend \(Entregable #2\)](#)
 - [1. Diagrama de componentes \(contenedores y capas\)](#)
 - [2. Diagrama de clases — slice de Screening/Diagnóstico](#)
 - [3. Diagrama de clases — Inyección de dependencias y seguridad](#)
 - [4. Despliegue \(Railway, 1 proyecto\)](#)
- [Cifrado de Datos Sensibles — CogniFit \(Entregable #4\)](#)
 - [1. Patrón \(igual al pizarrón\)](#)
 - [2. Diagrama de clases](#)
 - [3. Flujo de escritura y lectura](#)
 - [4. Motor criptográfico](#)
 - [5. Cómo se integra en un repositorio](#)
 - [6. Verificación \(ejecutada\)](#)
 - [7. Relación con el cifrado ya existente \(pgcrypto\)](#)

Diseño del Backend — Implementación ↔ Patrón ↔ Requerimiento (Entregable #1)

Respalda el avance de **API y Base de Datos** con las decisiones de diseño y los patrones aplicados, enlazado a la tabla de trazabilidad (ADR) y a las Historias de Usuario (HU). Ámbito: parte de Rodrigo (Backend SOA / BD).

1. Arquitectura general

Backend **FastAPI** con arquitectura **hexagonal / por capas**:

- **API** (`api/api/v1/*`): routers por dominio + inyección de dependencias + middleware.
- **Aplicación** (`application/use_cases` , `application/services`): orquestación (Commands) y servicios.
- **Dominio** (`domain/`): *ports* (interfaces), entidades y *value objects*.
- **Infraestructura** (`infrastructure/`): repositorios PostgreSQL, clientes de los microservicios PLN, seguridad (JWT, Argon2, pipeline de cifrado, auditoría), Redis.

Diagramas: ver [DISEÑO_API.md](#).

2. Mapa Implementación ↔ Patrón ↔ ADR/HU

Implementación (archivo/módulo)	Patrón	ADR / HU
dependencies/auth.py (get_current_user , JWT)	Proxy de autenticación	ADR-15 / HU-BK-01
App FastAPI + prefijo /api/v1 + middleware	Gateway (punto de entrada único)	ADR-16 / HU-BK-02
UserRepository , admin/router.py (CRUD)	Repository	ADR-17 / HU-BK-03
pg_student_repository.py (consulta por grupo/grado)	Query Object	ADR-18 / HU-BK-04
Use cases SubmitAnswersUseCase , GetResultUseCase	Command	ADR-19 / HU-BK-05
pg_tracking_repository.py (evaluate_progress , alertas)	Observer	ADR-20 / HU-BK-09
GenerateReportUseCase (ReportLab)	Builder	ADR-21 / HU-BK-10
infrastructure/cache (Redis)	Decorator de cache	ADR-22 / HU-BK-11
health.py (/health , /health/db , /health/pln)	Singleton / registro de salud	ADR-23 / HU-BK-12
PgModelVersionRepository.activate (estados del modelo)	State	ADR-24 / HU-BK-13
security/audit/audit_logger.py (append-only)	Decorator de auditoría	ADR-25 / HU-BK-14
UserRepository (usuarios + roles)	Repository	ADR-26 / HU-BD-01
get_db() (transacción por request)	Unit of Work	ADR-27 / HU-BD-03
DiagnosisServiceClient / RecommendationServiceClient	Facade / Adapter a microservicios	(consume ADR-12/13)
dependencies/services.py (@lru_cache)	Singleton de clientes/config	—
application/dtos/* , schemas Pydantic	DTO	ADR-10 / HU-MD-10
security/encryption/pipeline/* (AES-GCM)	Pipe & Filters / Chain	HU-BD-11

3. Modelo de datos (BD) y HU cubiertas

PostgreSQL con schemas por dominio: auth, academic, assessment, diagnosis, intervention, tracking, reporting, audit . Cobertura:

HU-BD	Implementación
HU-BD-01 Usuarios y roles	auth.users (Argon2, correo único, rol catálogo)
HU-BD-02 Estructura escolar	academic.schools/groups/students (FK, borrado lógico)
HU-BD-03 Tests y respuestas originales	assessment.tests/test_items/test_sessions/student_responses
HU-BD-04 Persistencia del diagnóstico	diagnosis.diagnoses (subtipo, severidad, riesgo, confianza)
HU-BD-05/06 Banco de ejercicios y rutas	intervention.exercises/route_templates/student_paths
HU-BD-07 Series temporales	tracking.diagnosis_ml_sessions

HU-BD	Implementación
HU-BD-08 Alertas	<code>tracking.alerts</code>
HU-BD-09 Reportes	<code>reporting.report_requests</code>
HU-BD-10 Versiones de modelo + auditoría	<code>diagnosis.ml_model_versions</code> , <code>audit.audit_log</code> (append-only)
HU-BD-11 Protección de datos de menores	<code>pgcrypto</code> (full_name) + pipeline de cifrado AES-GCM (app-side) + RLS + consentimiento

Migraciones: `schema.sql` → 002 → 003 → 004 → 005 (ver `deploy/`).

4. Seguridad (OWASP)

- **RBAC** por rol (`require_roles`); **RLS** preparado (`app.current_user_id/role`).
- **JWT** access corto + refresh revocable; **Argon2id** para contraseñas.
- **Cifrado de PII**: `pgcrypto` en BD + pipeline AES-GCM en aplicación (HU-BD-11).
- **Auditoría** append-only; **rate limiting**; cabeceras de seguridad (CSP, HSTS, etc.).

5. Integración con microservicios ML (Carlos)

El backend consume Diagnosis (8001) y Recommendation (8002) vía clientes con **fallback local** si no responden (`PLN_FALLBACK_ENABLED`). Contrato y flujo en `docs/flujo_completo.md` y `Pln/integracion_rodrigo.md` .

6. Estado de avance

- API: 9 dominios de endpoints implementados y desplegados (Railway + Neon).
- Flujo de test de punta a punta operativo (sesión → ítems → respuestas → diagnóstico → ruta).
- Cifrado de datos sensibles (patrón de clase) implementado y probado — ver [CIFRADO_DATOS_SENSIBLES.md](#).
- 40 requerimientos proyectados en la tabla de trazabilidad (entregable #3).

Diseño Visual de la API — CogniFit Backend (Entregable #2)

Diagramas de la implementación entregada del backend (FastAPI, arquitectura hexagonal / por capas). Render con Mermaid.

1. Diagrama de componentes (contenedores y capas)

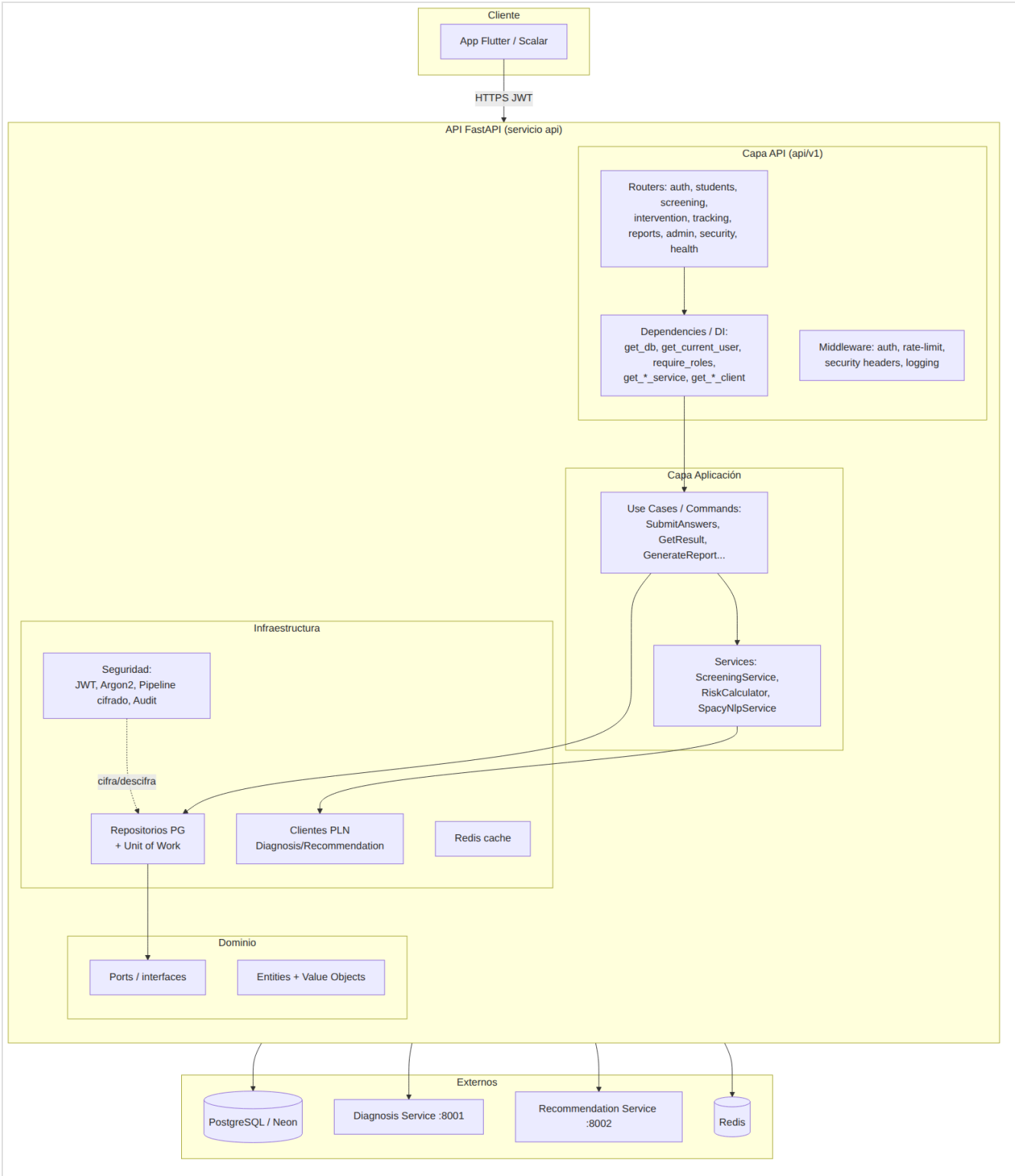


Figura: Diagrama de componentes

2. Diagrama de clases — slice de Screening/Diagnóstico

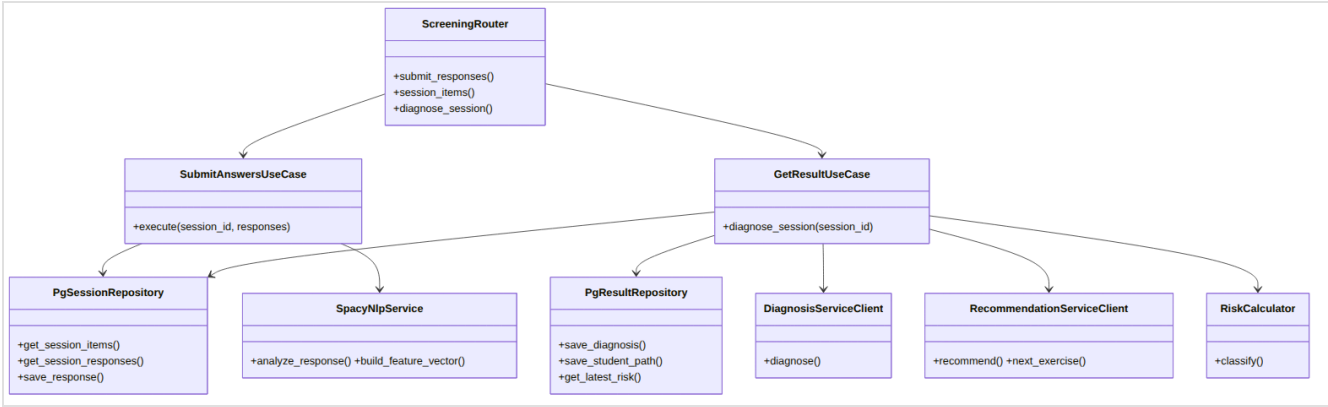


Figura: Clases — flujo de screening/diagnóstico

3. Diagrama de clases — Inyección de dependencias y seguridad

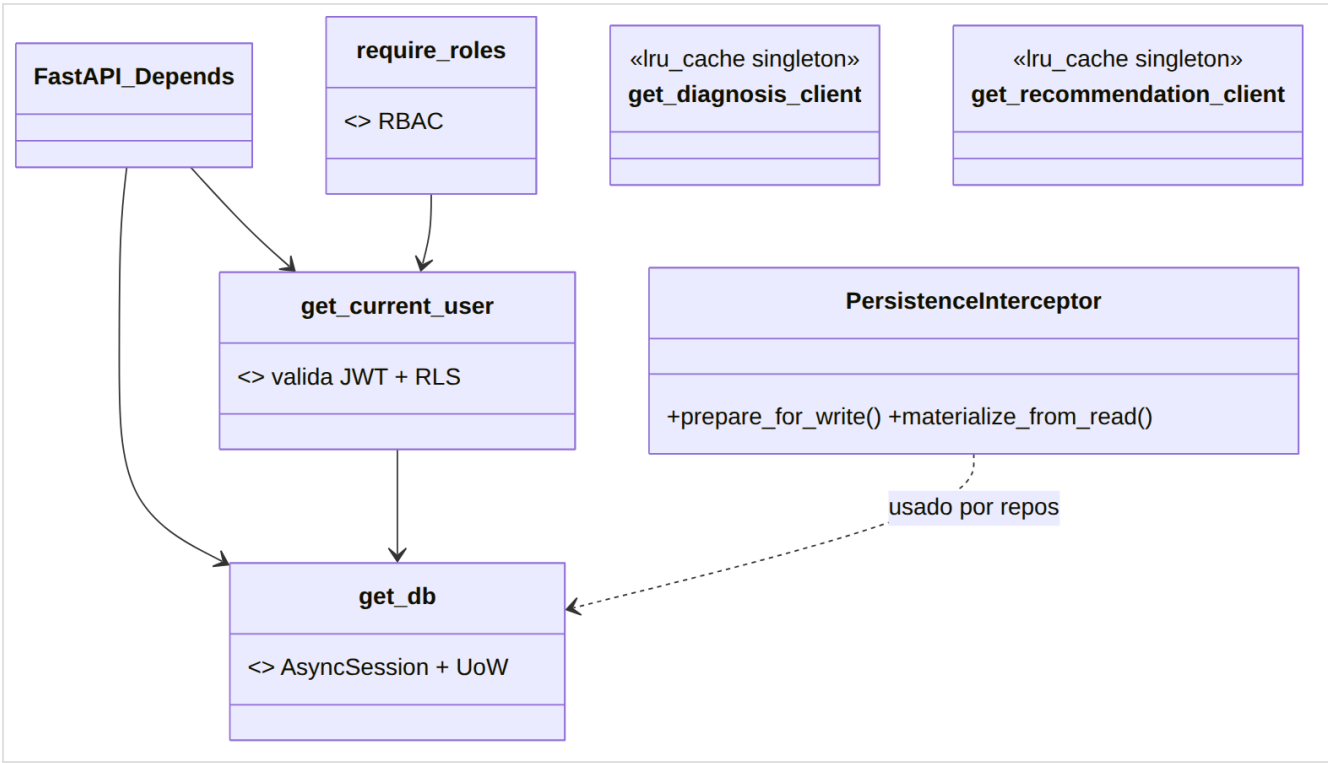


Figura: Clases — inyección de dependencias y seguridad

4. Despliegue (Railway, 1 proyecto)

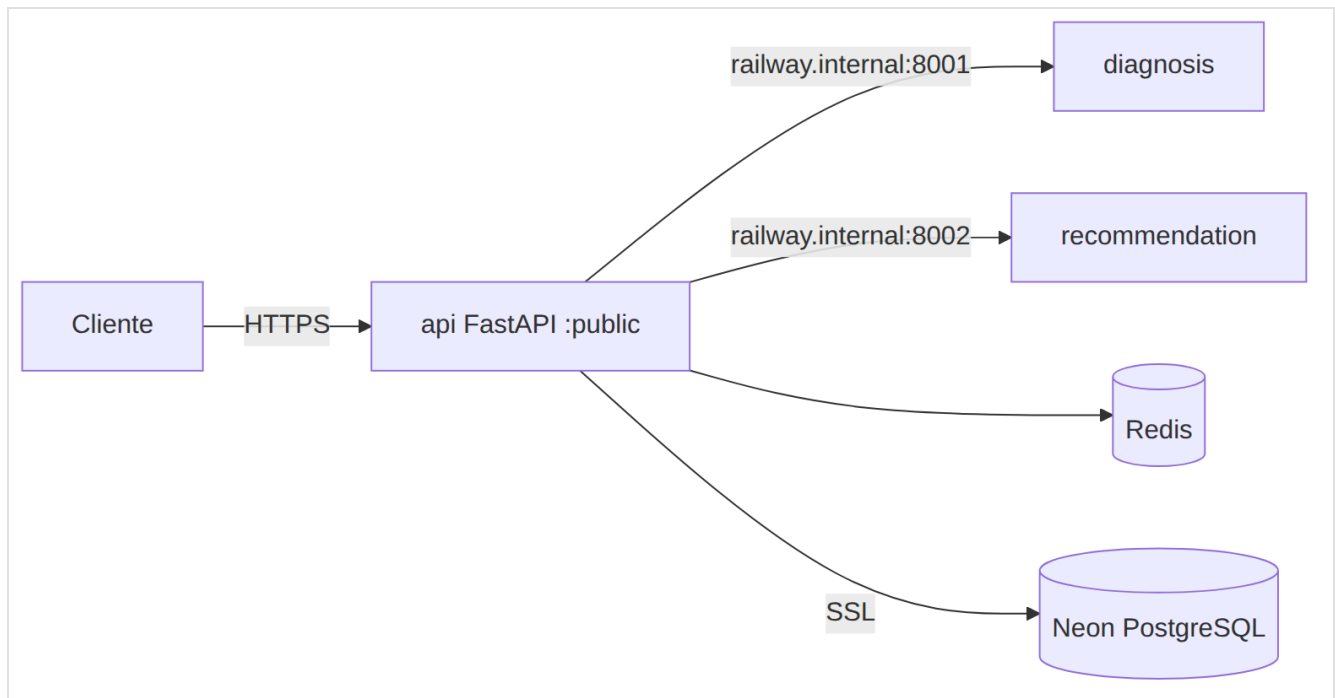


Figura: Despliegue (Railway + Neon)

Detalle de cada componente y sus patrones en [DOC_DISEÑO_BACKEND.md](#). El módulo de cifrado tiene su propio diagrama de clases en [CIFRADO_DATOS_SENSIBLES.md](#).

Cifrado de Datos Sensibles — CogniFit (Entregable #4)

Implementación del **proceso de cifrado en reposo / descifrado en uso** visto en clase, aplicado al backend. Cubre **HU-BD-11** (protección de datos de menores) y **HU-BK-14 / HU-BD-10** (no exponer PII).

Endurecimiento respecto al demo: el motor usa **AES-256-GCM** (cifrado autenticado) en lugar del XOR de stream del ejemplo. La clave se deriva **por scope** y el scope se liga como **AAD**, de modo que un ciphertext de una tabla no es válido en otra.

1. Patrón (igual al pizarrón)



- **@sensible** ([decorators.py](#)): marca declarativa de los campos a cifrar (metadata de dataclass).
- **MetadataEngine** ([metadata_engine.py](#)): descubre y cachea los campos sensibles por tipo.
- **PersistenceInterceptor** ([interceptor.py](#)): punto único; cifra antes de persistir, descifra al leer. Trabaja sobre una **copia** (no muta el objeto de dominio).
- **CipherPipe / DecryptPipe** ([pipes/](#)): cadena de filtros (PIPE). Filtros intercambiables.
- **KeyProvider** ([key_provider.py](#)): deriva clave AES-256 por scope; rechaza claves inseguras en producción.

2. Diagrama de clases

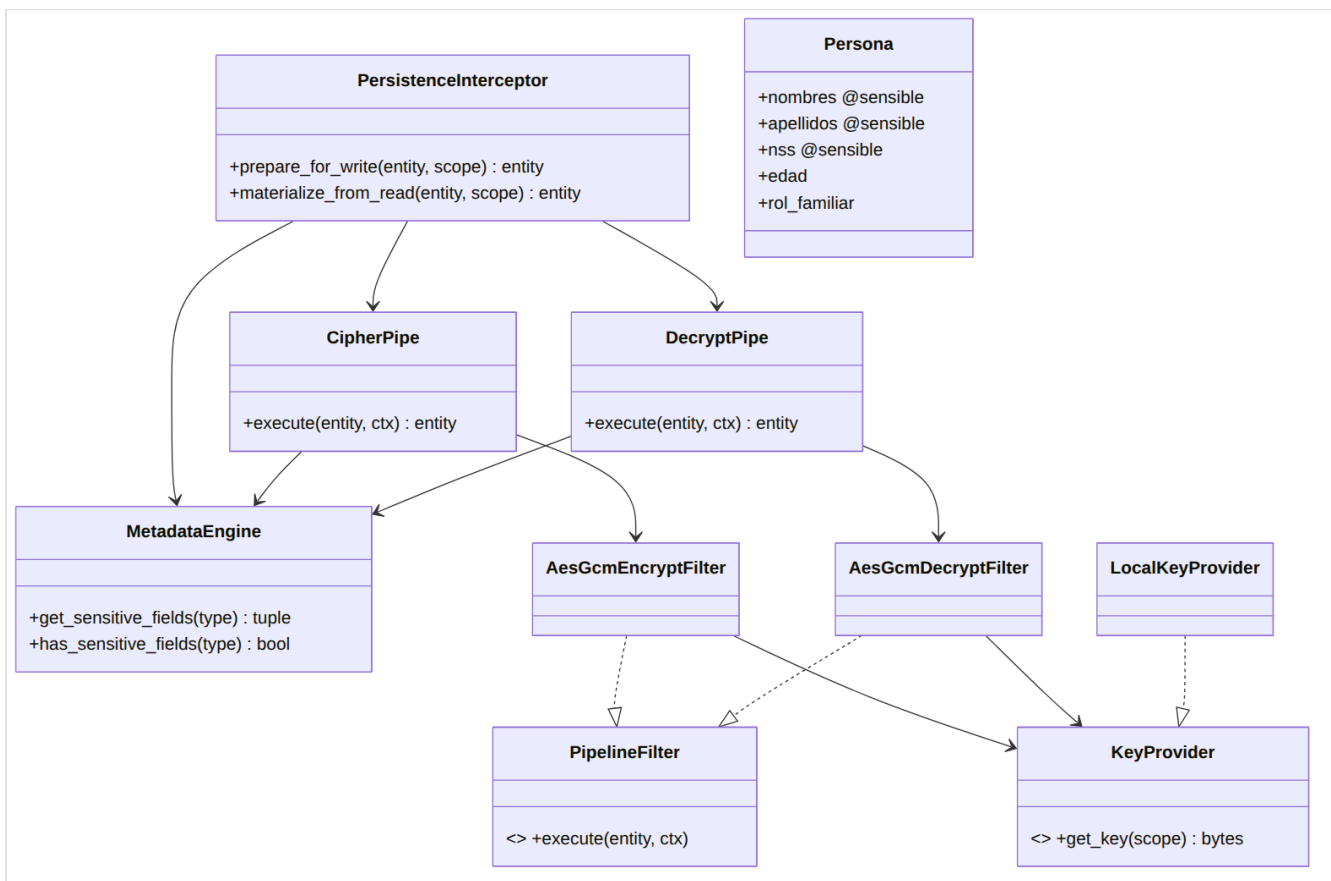


Figura: Clases del pipeline de cifrado

3. Flujo de escritura y lectura

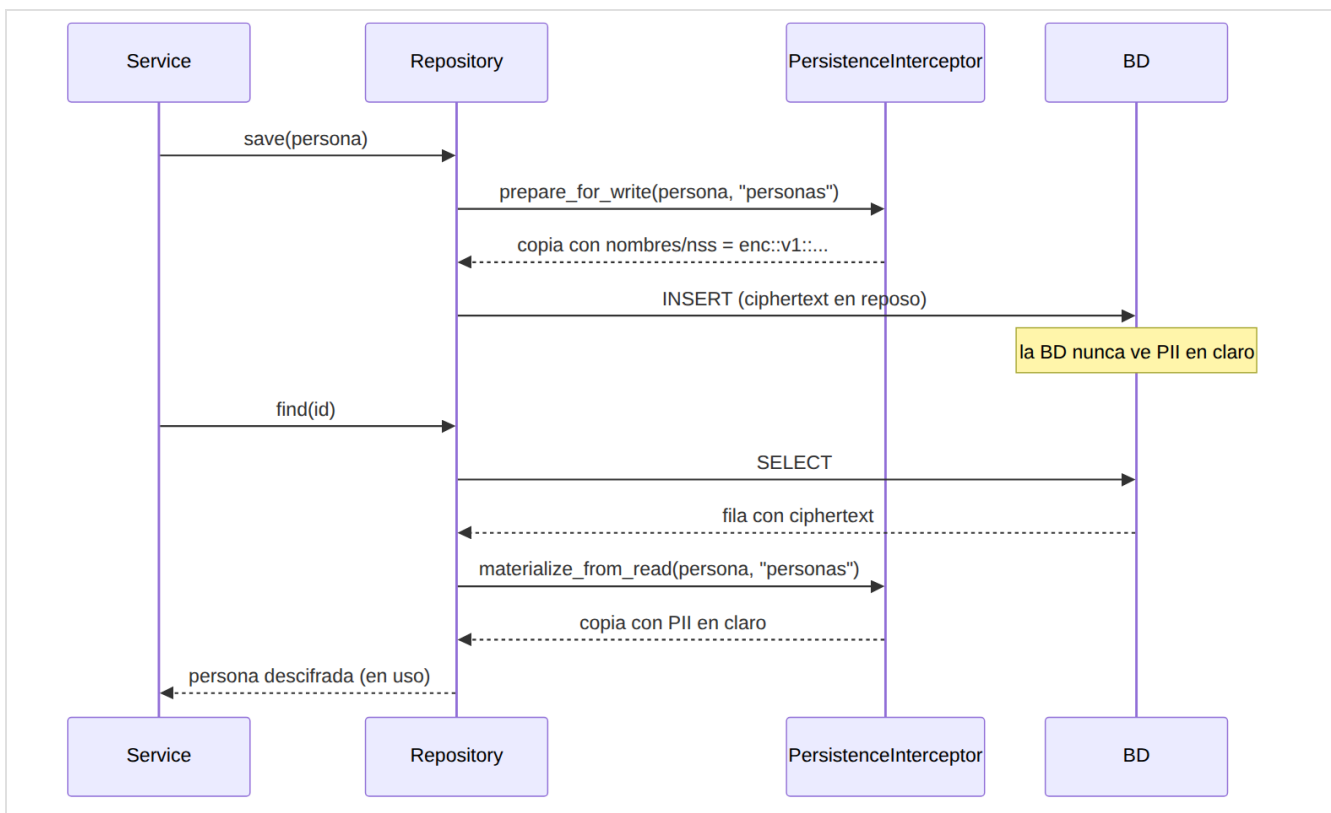


Figura: Secuencia cifrar/descifrar

4. Motor criptográfico

- **Algoritmo:** AES-256-GCM (cryptography), autenticado (integridad + confidencialidad).
- **Clave:** derivada por scope — SHA-256(master || ':' || scope) (32 bytes). La *master key* sale de FIELD_ENCRYPTION_KEY (o DB_ENCRYPTION_KEY); en un entorno real vendría de un KMS.
- **Nonce:** 12 bytes aleatorios por valor (dos cifrados del mismo texto dan tokens distintos).
- **AAD:** el scope (tabla) → un ciphertext no se puede mover de tabla.
- **Formato:** enc::v1::base64url(nonce || ciphertext+tag) . Idempotente (no recifra) y resiliente (ignora valores ya en claro al leer).

5. Cómo se integra en un repositorio

```
class PersonaRepository:
    def __init__(self, interceptor: PersistenceInterceptor) -> None:
        self._itc = interceptor

    def save(self, persona: Persona) -> Persona:
        at_rest = self._itc.prepare_for_write(persona, scope="personas")
        # ... INSERT con at_rest.nombres (ya cifrado) ...

    def find(self, id) -> Persona:
        row = ... # SELECT
        return self._itc.materialize_from_read(row_as_persona, scope="personas")
```

El interceptor se obtiene con build_persistence_interceptor() (lee la config) y se inyecta por DI como el resto de servicios.

6. Verificación (ejecutada)

Pruebas en [test_encryption_pipeline.py](#): cifrado solo de campos sensibles, round-trip, no-mutación del original, idempotencia, fallo con scope incorrecto (AAD) y rechazo de clave insegura en producción. **Todas pasan.**

7. Relación con el cifrado ya existente (pgcrypto)

Hoy academic.students.full_name se cifra **a nivel de BD** con pgcrypto (pgp_sym_encrypt/decrypt). Ese es cifrado-en-reposo del lado servidor de Postgres. Este pipeline es el patrón **del lado de aplicación** (el enseñado en clase): desacopla el cifrado del SQL, es declarativo (@sensible) y portable a cualquier almacenamiento. Es el mecanismo recomendado para nuevas entidades con PII y puede sustituir progresivamente al cifrado inline.